

Software Requirements Engineering (SWE3104)

Instructor: Behailu Getachew (Asst. Prof.)



Chapter 3- Developing Requirement Analysis



Week 4 and 5 Lessons:

Developing Requirement Analysis

- ❖ In summary, the process of requirement **elicitation consists of** steps to
 - ❖ **Identify actors**
 - ❖ **Identify scenarios**
 - ❖ **Identify use cases**
 - ❖ **Refine use cases**
 - ❖ **Identify relationship between actors and use cases**
 - ❖ **Identify initial object analysis**
 - ❖ **Identify non-functional requirements**



Week 4 and 5 Lessons: Developing Requirement Analysis

❖ **Revision Example: Buy Drink Use Case**

Use Case: Buy Drink

Scope: Vending Machine

Level: User Goal

Intention in Context: The intention of the *Customer* is to buy a drink in exchange of money.

Multiplicity: There can always be only one *Customer* interacting with the system at a given time.

Primary Actor: Customer

Secondary Actors: Selector Button, Coin Slot, Shelf, Sensor, Money Box, Drink Light, Cancel Button, Display, Terminal

Precondition: The system is in service, filled with drinks and change, and the Money Box is not full.



Week 4 and 5 Lessons:

Developing Requirement Analysis

❖ **Revision Example:** Buy Drink Use Case

Main Success Scenario:

Customer selects drink by pushing appropriate drink selector button.

1. *Button* notifies *System* of selected drink.
2. *System* displays the price of the selected drink on *Display*.

Customer inserts a coin into Coin Slot.

3. *Coin Slot* notifies *System*.
 4. *System* recognizes the coin, and updates the remaining price on *Display*.
- Steps 3 and 4 are repeated until the amount of inserted money reaches or exceeds the price of the drink.*
5. *System* validates that there are sufficient funds for the selection and notifies *Shelf* to start dispensing the drink.
 6. *Sensor* informs *System* that the drink has been dispensed.
 7. *System* asks *Money Box* to collect the specified amount of money and, if necessary, provide the change.

Customer collects the drink and optionally the change.



Week 4 and 5 Lessons:

Developing Requirement Analysis

❖ **Revision Example: Buy Drink Use Case**

Extensions:

2a. *System* ascertains that the selected drink is not available and flashes *Drink Lights*; use case ends in failure.

4a. *System* fails to identify the coin; *System* asks *Money Box* to eject coin; use case continues at step 3.

(3-4)a. *Customer* informs *System* to abort the sale by hitting the *Cancel* button;

(3-4)a.1 *System* asks *Money Box* to eject coins; use case ends in success.

(3-4)b. *System* times out.

(3-4)b.1 *System* asks *Money Box* to eject the inserted coins; use case ends in failure.

5a. *System* ascertains that the inserted money exceeds the price for the drink and that there is not enough change;

5a.1 *System* asks *Money Box* to eject inserted coins.

5a.2 *System* displays “no change” on *Display*; use case ends in failure.

7a II. The *Money Box* is full.

7a II.1 *System* displays “no service” on *Display* and goes out of service; use case ends successfully.

7b II. The delivered drink was the last one of that kind.

7b II.1 *System* turns on the appropriate *Drink Light*; use case ends successfully.



Week 4 and 5 Lessons:

Developing Requirement Analysis

❖ **Revision Example: Service Machine Use Case**

Use Case: Service Machine

Scope: Vending Machine

Level: User Goal

Intention in Context: The intention of the Service Person is to maintain the machine by ensuring that there are drinks available, modifying drink pricing, and by collecting the money earned.

Multiplicity: There can be only one service person servicing the machine at a given time.

Primary Actor: Service Person

Precondition: No Customer is currently using the system.



Week 4 and 5 Lessons: Developing Requirement Analysis

❖ **Revision Example:** Buy Drink Use Case

Main Success Scenario:

Service Person interacts with the system by using the Terminal.

1. *Service Person* identifies himself with *System*.

Steps 2-3 can be repeated for each shelf, in any order.

2. *Service Person* informs *System* of new price for a shelf.

3. *Service Person* replenishes a shelf and informs *System* of new number of drinks for that shelf.

4. *Service Person* empties the *Money Box*, replenishes the change and informs the *System*.

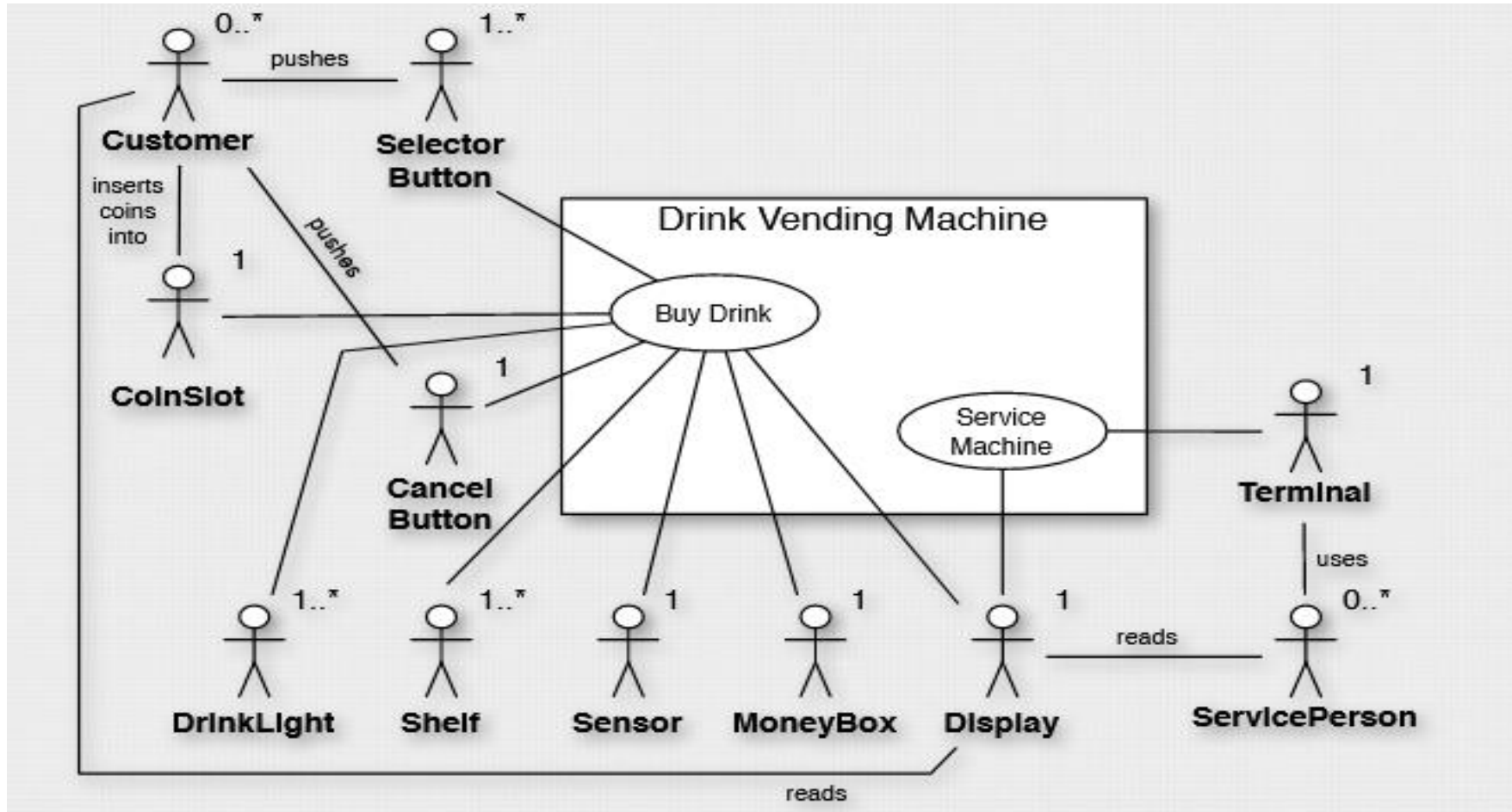
5. *Service Person* informs *System* that maintenance is over.

Extensions:

2a. *System* fails to identify the *Service Person*; use case ends in failure.

Week 4 and 5 Lessons: Developing Requirement Analysis

❖ Revision Example: Vending Machine Use Case Diagram





Week 4 and 5 Lessons: Developing Requirement Analysis

❖ **Home Work:** Drink Vending Machine Questions

○ **URN: User Requirement Notation.**

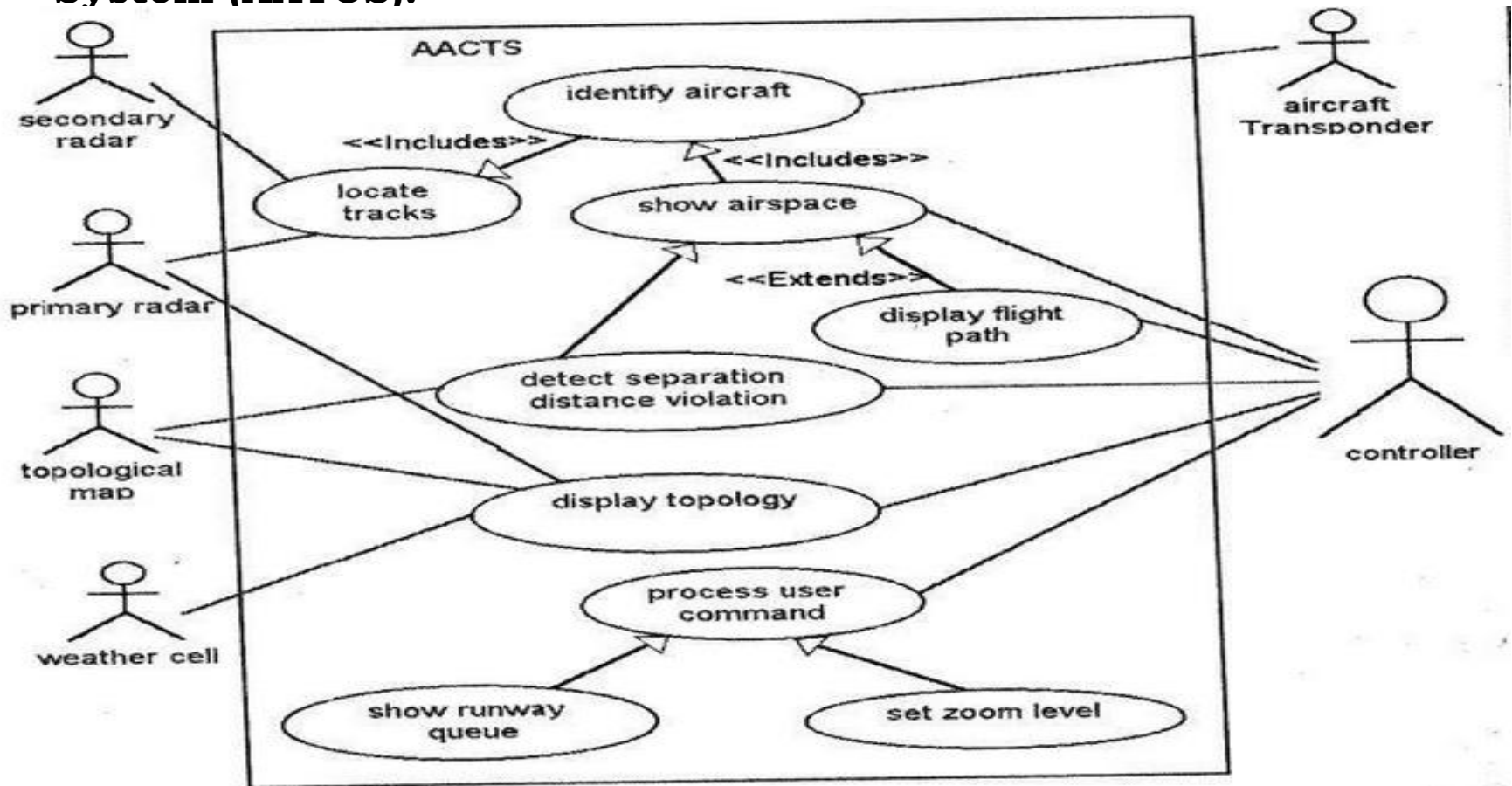
1. Create a URN model for the BuyDrink use case.

- If you decide to group several basic interaction steps into one URN responsibility, please use the description field of the responsibility to document which use case steps it represents.

2. Create a URN model for the ServiceMachine use case.

Week 4 and 5 Lessons: Developing Requirement Analysis

❖ **Home Work:** Use Case Model for Automated Air Traffic Control System (AATCS).





Week 4 and 5 Lessons: Developing Requirement Analysis

- ❖ **Home Work:** Consider AATCS use case model to do the following tasks:
1. **Derive the Use Case Descriptions.**
 2. **Suggest and describe at least one main use case scenarios.**
 3. **Describe the extension (s).**



Week 4 and 5 Lessons:

Developing Requirement Analysis

- ❖ ISO/IEC/IEEE 29148 Standard for Requirements Analysis.
 - Requirements Specification:
 - Requirements document structure
 - Requirements document standard and Guidelines
- ❖ (C)lass (R)esponsability (C)ollaborators cards.
 - Linking Requirements to UML Analysis Models
 - Examples: Case Studies

Week 4 and 5 Lessons:

Developing Requirement Analysis



- **Requirement Analysis**

- **What is it?**

- The process by which **customer needs** are understood and documented.
 - Expresses “**what**” is to be built and **NOT** “**how**” it is to be built.

- **Example 1:**

- The system shall allow users to withdraw cash. [**What?**]

- **Example 2:**

- A sale item's name and other attributes will be stored in a hash table and updated each time any attribute changes. [**How?**]

- **How to make a standard requirement analysis?**

- Using ISO/IEC/IEEE 29148:2018 standard formulation.

- **How to develop the analysis for ensuring such standard?**

- **Using Requirement Analysis Tool (RAT).**

- jUCMNav Tool, ReqIF studio, QUARCC, QuARS, KaOS.

Week 4 and 5 Lessons:

Developing Requirement Analysis



- ❖ ISO/IEC/IEEE 29148 Standard for Requirements Analysis.
 - ISO/IEC/IEEE 29148 Requirement lifecycle: Requirements engineering follows this document to define the standard requirement engineering processes for development of software and hardware products.
 - The basic standard for specifying the software requirements is ISO/IEC/IEEE 29148:2018, which regulates the structure and required items of the specification.
 - The known models, methods and tools analyzed do not enable to solve the problem of software requirements specification analysis on its structure correctness (according to ISO/IEC/IEEE 29148:2018).



Week 4 and 5 Lessons: Developing Requirement Analysis

- ❖ ISO/IEC/IEEE 29148 Standard for Requirements Analysis.
 - The formalization of the structure of software requirements specification in the form of **set-theoretical models** of **sections of the specification** has to be defined (according to ISO/IEC/IEEE 29148:2018).
 - How to represent the formalized form of software requirements specification on its structure correctness (according to ISO/IEC/IEEE 29148:2018)?
 - Given the structure of the specification of the software requirements in accordance with ISO 29148:2018, let's represent the specification in the following formalized form:

$$\mathbf{SRS} = \langle \mathbf{S_1}, \dots, \mathbf{S_19} \rangle,$$

Week 4 and 5 Lessons: Developing Requirement Analysis



- ❖ ISO/IEC/IEEE 29148 ...: **SRS=<S_1,...,S_19>**,
 - where **S_1** – section “**Purpose**” of the software requirements specification, **S_2** – section “**Scope**”, **S_3** – section “**Product perspective**”, **S_4** – section “**Product functions**”, **S_5** – section “**User characteristics**”, **S_6** – section “**Limitations**”, **S_7** – section “**Assumptions and dependencies**”, **S_8** – section “**Apportioning of requirements**”, **S_9** – section “**Specific requirements**”, **S_10** – section “**External interfaces**”, **S_11** – section “**Functions**”, **S_12** – section “**Usability requirements**”, **S_13** – section “**Performance requirements**”, **S_14** – section “**Logical database requirements**”, **S_15** – section “**Design constraints**”, **S_16** – section “**Standards compliance**”, **S_17** – section “**Software system attributes**”, **S_18** – section “**Verification**”, **S_19** – section “**Supporting information**” of the software requirements specification.



Week 4 and 5 Lessons:

Developing Requirement Analysis

- ❖ ISO/IEC/IEEE 29148 Standard for Requirements Analysis.
 - **SRS=<S_1,...,S_19>**,
 - **SRS is Software Requirement Specification.**
 - **S_1** = {ps},
 - ps – purpose of software;
 - **S_2** = {isp, spd, spb, spo, spg, cssh},
 - isp – the software product's identifying,
 - spd– what software product will provide,
 - spb – benefit of the software product,
 - spo – objectives of the software product,
 - spg – goals of the software product,
 - cssh – consistency with the same statements in high-level specifications;



Week 4 and 5 Lessons:

Developing Requirement Analysis

- ❖ ISO/IEC/IEEE 29148 Standard for Requirements Analysis.
 - **SRS=<S_1,...,S_19>**,
 - **S_3** = {srrp, sosi, soui, sohi, soswi, soci, som, soo, sosar},
 - srrp – software system's relationship to other related products,
 - sosi – software operation within the system interfaces,
 - soui – software operation within the user interfaces,
 - sohi – software operation within the hardware interfaces,
 - soswi – software operation within the software interfaces,
 - soci – software operation within the communications interfaces,
 - som – software operation within the memory,
 - soo – software operation within the operations,
 - sosar – software operation within the site adaption requirements



Week 4 and 5 Lessons: Developing Requirement Analysis

- ❖ ISO/IEC/IEEE 29148 Standard for Requirements Analysis.
 - **SRS**=<**S_1**,...,**S_19**>,
 - **S_4** = {mf},
 - mf– major future functions of the software.
 - **S_5** = {gcigu, gciu},
 - gcigu – general characteristics of the software product's groups of users,
 - gciu – general characteristics which influence usability;



Week 4 and 5 Lessons:

Developing Requirement Analysis

- ❖ ISO/IEC/IEEE 29148 Standard for Requirements Analysis.
 - **SRS=<S_1,...,S_19>**,
 - **S_6** = {rp, hwl, ioa, plo, af, cf, holr, shp, quar, ca, ssc, pmc},
 - rp – regulatory policies,
 - hwl –limitations of hardware,
 - ioa – interfaces to other applications,
 - plo – parallel operation,
 - af – audit functions,
 - cf – control functions,
 - holr – requirements of higher-order language,
 - shp – protocols of signal handshake,
 - quar – quality requirements,
 - ca – criticality of the application,
 - ssc – considerations of safety and security,
 - pmc – physical/mental considerations



Week 4 and 5 Lessons: Developing Requirement Analysis

- ❖ ISO/IEC/IEEE 29148 Standard for Requirements Analysis.
 - **SRS**=<**S_1**,...,**S_19**>,
 - **S_7** = {far},
 - far – factors which influence the requirements;
 - **S_8** = {asrse, crtff, crte, rduf},
 - asrse – distribution the requirements to elements of software,
 - crtff – cross-reference table by function,
 - crte – cross-reference table by element,
 - rduf – requirements that can be transferred in software's future versions;



Week 4 and 5 Lessons:

Developing Requirement Analysis

- ❖ ISO/IEC/IEEE 29148 Standard for Requirements Analysis.
 - **SRS**=<**S_1**,...,**S_19**>,
 - **S_9** = {rlds, iss, oss, fss},
 - rlds – requirements to a detail sufficient level,
 - iss – inputs into the software system,
 - oss – outputs from the software system,
 - fss – functions of the software system in response to the input or in support of the output;



Week 4 and 5 Lessons:

Developing Requirement Analysis

- ❖ ISO/IEC/IEEE 29148 Standard for Requirements Analysis.
 - **SRS=<S_1,...,S_19>**,
 - **S_10** = {ni, dp, sido, vrat, um, tg, roio, sfo, wfo, df, cmf, emsg},
 - ni – item's name,
 - dp – purpose's description,
 - sido – input's source or output's destination,
 - vrat – valid range, accuracy, and/or tolerance,
 - um – measuring units,
 - tg – timing,
 - roio – relationships to other inputs/outputs,
 - sfo – formats/organization of screen,
 - Wfo – formats/organization of window,
 - df – formats of data,
 - cmf – formats of command,
 - emsg – end messages;



Week 4 and 5 Lessons:

Developing Requirement Analysis

- ❖ ISO/IEC/IEEE 29148 Standard for Requirements Analysis.
 - **SRS=<S_1,...,S_19>**,
 - **S_11** = {faapi, fapgo, vci, eso, ras, ep, rsoi},
 - faapi – basic actions that must take place in the software when receiving and processing inputs,
 - fapgo – basic actions that must occur in the software when processing and generating outputs,
 - vci – checks of validity on the inputs,
 - eso –exact sequence of operations,
 - ras – responses to abnormal situations,
 - ep – parameters' effect,
 - rsoi – relationship of inputs and outputs;

Week 4 and 5 Lessons: Developing Requirement Analysis



- ❖ ISO/IEC/IEEE 29148 Standard for Requirements Analysis.
 - **SRS=<S_1,...,S_19>**,
 - **S_12** = {ubr, mec, mefc, msc},
 - ubr – usability requirements,
 - mec – measurable criteria of effectiveness in specific use contexts,
 - mefc– measurable criteria of efficiency in specific use contexts,
 - msc– measurable criteria of satisfaction in specific use contexts;



Week 4 and 5 Lessons:

Developing Requirement Analysis

- ❖ ISO/IEC/IEEE 29148 Standard for Requirements Analysis.
 - **SRS=<S_1,...,S_19>**,
 - **S_13** = {snr, dnr, nts, nssu, athi, nota, not, adnw, adpw, prq},
 - snr – static numerical requirements,
 - dnr – dynamic numerical requirements,
 - nts – number of supported terminals,
 - nssu – number of supported simultaneous users,
 - athi – amount and type of handled information,
 - nota – numbers of transactions,
 - not – number of tasks,
 - adnw – amount of the processed data within certain time periods for normal workload conditions,
 - adpw – amount of the processed data within certain time periods for peak workload conditions,
 - prq – requirements of performance



Week 4 and 5 Lessons:

Developing Requirement Analysis

- ❖ ISO/IEC/IEEE 29148 Standard for Requirements Analysis.
 - **SRS=<S_1,...,S_19>**,
 - **S_14** = {tiuvf, fqu, asc, der, ics, drr},
 - tiuvf – types of used information,
 - fqu – frequency of use,
 - asc – accessing the capabilities,
 - der – entities of data and their relationships,
 - ics – constraints of integrity,
 - drr – requirements of data retention;
 - **S_15** = {ces, crr, cpl},
 - ces – the software system design's constraints which are imposed by external standards,
 - crr – the software system design's constraints which are imposed by regulatory requirements,
 - cpl – the software system design's constraints which are imposed by project limitations;



Week 4 and 5 Lessons:

Developing Requirement Analysis

- ❖ ISO/IEC/IEEE 29148 Standard for Requirements Analysis.
 - **SRS**=<**S_1**,...,**S_19**>,
 - **S_16** = {rf, dn, ap, at},
 - rf – report format,
 - dn –data naming,
 - ap – accounting procedures,
 - at – audit tracing



Week 4 and 5 Lessons:

Developing Requirement Analysis

- ❖ ISO/IEC/IEEE 29148 Standard for Requirements Analysis.
 - **SRS=<S_1,...,S_19>**,
 - **S_17** = {rb, avb, scr, cct, slhd, fdm, csa, dicv, dp, mb, pb, pehdc, pchd, uppl, upcls, upos},
 - rb – reliability,
 - avb –availability,
 - scr – security,
 - cct – certain techniques of cryptographic,
 - slhd – data sets of specific log or history,
 - fdm – certain functions to different modules,
 - csa – communications between some areas of the software product,
 - dicv – integrity of data for critical variables,
 - dp – privacy of data,
 - mb – maintainability,



Week 4 and 5 Lessons:

Developing Requirement Analysis

- ❖ ISO/IEC/IEEE 29148 Standard for Requirements Analysis.
 - **SRS=<S_1,...,S_19>**,
 - **S_17** = {rb, avb, scr, cct, slhd, fdm, csa, dicv, dp, mb, pb, pehdc, pchd, uppl, upcls, upos},
 - pb – portability,
 - pehdc – the percentage of elements with host-dependent code,
 - pchd – the percentage of host dependent code,
 - uppl – portable language use,
 - upcls – particular compiler or language subset use,
 - upos – operating system use;



Week 4 and 5 Lessons:

Developing Requirement Analysis

- ❖ ISO/IEC/IEEE 29148 Standard for Requirements Analysis.
 - **SRS**=<**S_1**,...,**S_19**>,
 - **S_18** = {va, mqs},
 - va – approaches for verification,
 - mqs – methods for qualifying the software;
 - **S_19** = {siof, dcas, rus, sbi, dpss, spi},
 - siof – sample input/output formats,
 - dcas – cost analysis studies' descriptions,
 - rus – user surveys' results,
 - sbi – supporting or background information that can help the readers of the SRS,
 - dpss – description of the problems which will be solved by the software,
 - spi – special packaging instructions for the code and the media for security, export, initial loading.



Week 4 and 5 Lessons: Developing Requirement Analysis

- ❖ How to discover whether an ideal and a real ontology analysis that supports the correct requirement specification or not?
 - The equations **S_1 - S_19** represent **all the necessary items** of the **software requirements specification** in terms of ISO/IEC/IEEE 29148:2018, **structured by the sections of the specification.**
 - For the structure of the specification of requirements to be recognized as correct, it is necessary that the **specification contains all the listed items.**
 - For **ensuring all items availability**, it is proposed to develop an **ideal ontology** based on the equations **S_1 - S_19** as well as to **develop a real ontology** based on each analyzed specification.
 - A **comparison** of the **two ontologies (ideal and real)** should result in the correct requirement **items of specification being set.**

Week 4 and 5 Lessons:

Developing Requirement Analysis

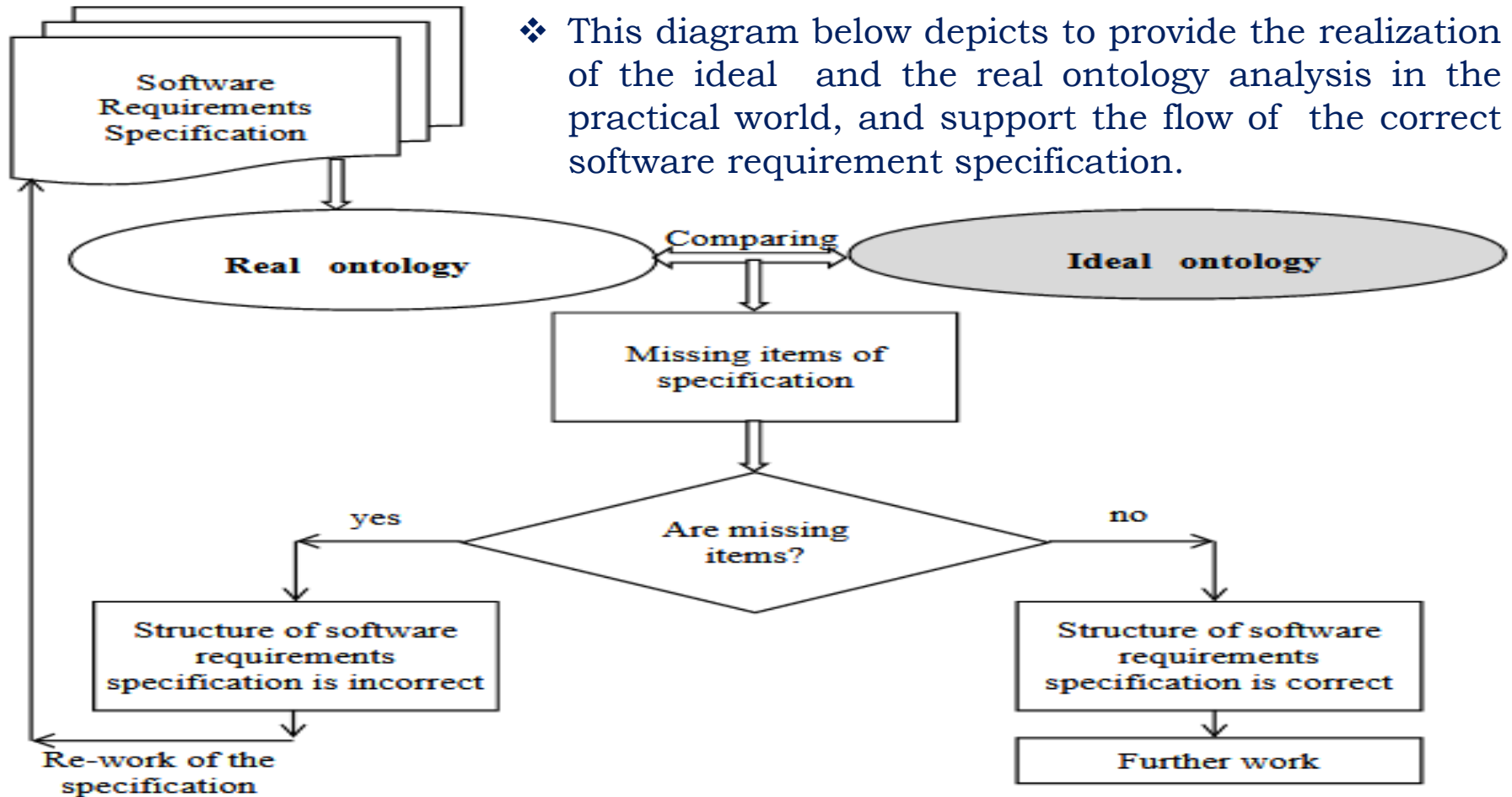


- ❖ How to discover whether an ideal and a real ontology analysis that supports the correct requirement specification or not ...?
 - **Case 1:** If either of the availability of items are missed, then the structure of specification is incorrect and re-work of the specification is proposed.
 - **Case 2:** If there are no such missing items, then the structure of specification is correct and further work is proposed.
 - The use of ontologies in this approach to the analysis of **software requirements specification on its structure correctness** provides **the automation of analysis** (according to ISO/IEC/IEEE 29148:2018).

Week 4 and 5 Lessons: Developing Requirement Analysis

❖ Real vs Ideal Ontology Comparison Diagram:

- ❖ This diagram below depicts to provide the realization of the ideal and the real ontology analysis in the practical world, and support the flow of the correct software requirement specification.





Week 4 and 5 Lessons:

Developing Requirement Analysis

❖ Requirements Specification Document (RSD)

- This document describes each of the **essential requirements (functions, performance, design constraints, and quality attributes)** of the **system / software** and its **external interfaces**.
- Defines the **scope** and **boundaries** of the **system / software**.
- **Elaborated** from **elicitation** notes.
- **Reading Assignment:**
 - **IEEE 830 and IEEE 29148 Standards and Guidelines for formation of RSD.**
 - **Use these two IEEE standards for document structure and requirement specification guidelines.**

Week 4 and 5 Lessons: Developing Requirement Analysis



❖ Common Requirements Problems

- **Requirements that are easy to misunderstand.**
 - **Often, this problem is caused by the use of ambiguous terms, terminological inconsistencies or the use of non-standard syntaxes for documenting requirements.**
- **Use of ambiguous terms.** Consider the requirement “**The Payroll Database shall respond to all queries quickly**”. It is **unclear** what “**quickly**” means in this context (**1** ms, **1** sec, **10** sec, etc.) and this may lead to a **mismatch** between the **expectations of the stakeholder** and the **interpretation of the developer**, ultimately leading to **rework** and/or **customer dissatisfaction**.

Week 4 and 5 Lessons: Developing Requirement Analysis



❖ Common Requirements Problems ...

○ Often, this problem ...

- **Terminological inconsistency.** Consider the following requirements: 1) “The **Order Entry System** shall allow users to view **all orders placed in a day**” and 2) “The **Order Processing System** shall generate **daily reports**”. In this case, the **requirements analyst** uses **two different terms** to refer to the **same system**, leading to **confusion**.

Week 4 and 5 Lessons: Developing Requirement Analysis



❖ Common Requirements Problems ...

○ Often, this problem ...

- **Use of non-standard syntaxes such as missing agent.**
Consider the requirement “Shall have the **ability to generate profit reports**”. It is **unclear** which **agent** (e.g., which **system** or **sub-system**) shall have the ability to generate the profit reports. In this case, the **designers/developers** may try to **guess** which **system** should have that **functionality**, **potentially** leading to **rework** at a **later phase**.

Week 4 and 5 Lessons: Developing Requirement Analysis



❖ Common Requirements Problems ...

- **Requirements that are inconsistent.**
 - This problem is caused by requirements either **conflicting with each other** or **with some policy** or **business rule**.
 - **Example:** Consider the following **two requirements**: “The Payroll database shall authenticate **all requests** using **LDAP**” and “The payroll database must respond to all requests **within 1 millisecond**”. In this case, **it may not be feasible to satisfy** this requirement if the **LDAP server** takes **more than 1 millisecond to process each request**. If such a **defect** is **not detected early on**, then it is possible that the **infeasibility** is **only realized** when the **LDAP server** and the **Payroll database** have been **designed** and **implemented**, leading to **large costs** in **rework**.

Week 4 and 5 Lessons: Developing Requirement Analysis



❖ Common Requirements Problems ...

○ **Requirements that are incomplete.**

- This problem is caused by **missing some requirements** of a **certain type**.
 - **Example:** It is often the case that **performance requirements for a system are omitted** either due to the **lack of knowledge among the stakeholders** or **because the requirements analysts fail to elicit them**. This leaves the **technical designers** and **developers** to make **design choices** about **the software system**, which **may or may not** meet the **stakeholder's approval**.

Week 4 and 5 Lessons:

Developing Requirement Analysis



❖ Some practices to avoid common requirements problems ...

- **Use the best practices for writing requirements.**
 - Write complete sentences that have proper spelling and grammar.
 - Use active voice (For e.g., “ The **System shall send an e-mail**” instead of “**E-mail will be sent**”) .
 - Use terms consistently and as defined in the glossary.
 - Do not use different phrases to refer to the same thing. (For e.g., Do not use Order Processing System and Order Entry System to refer to the same system).
 - Write sentences in a **consistent fashion** starting with the **agent/actor**, followed by an **action** verb, followed by an **observable result**. (For e.g., “The System (**agent**) shall generate (**action** verb) a report (**result**)”).

Week 4 and 5 Lessons: Developing Requirement Analysis



❖ Some practices to avoid common requirements problems ...

- **Use the best practices for writing requirements**
 - Clearly specify the **trigger condition** that causes the system to perform a certain behavior. (For e.g., “If the **user** enters the **wrong password**, the system shall generate an **error message**”).

Week 4 and 5 Lessons: Developing Requirement Analysis



- ❖ **Techniques that should be used to avoid common requirements problems**
 - **Example: Syntactic and phrasal analysis: using Requirements Analysis Tool (RAT).**
 - **RAT** supports a **set of controlled syntaxes** for **writing requirements**. This set includes ..
 - **Standard Requirements Syntax.** This is the most commonly used syntax for writing requirements and is of the form:
 - **StandardRequirement: <agent> <modal word> <action> <rest>**

Week 4 and 5 Lessons: Developing Requirement Analysis



- ❖ **Techniques that should be used to avoid common requirements problems ...**
 - **Standard Requirements Syntax ...**
 - Where, **<agent>**, **<action>**, **<modal word>** are phrases in their respective **glossaries** and **<rest>** is the remainder of the sentence and can consist of agents, actions or any other words and is defined as:
 - **<rest>: [<anyword> | <agent> | <action>]***
 - **Example of standard requirement:** “The **system** **shall generate profit reports**.” Here “System” is the agent, “shall” is the modal word and “generate” is the action and “profit reports” is the **rest** of the requirement.

Week 4 and 5 Lessons: Developing Requirement Analysis



- ❖ **Techniques that should be used to avoid common requirements problems ...**
 - **Conditional Requirements Syntax.**
 - There are a **number of conditional requirements** supported by RAT.
 - For brevity, the most common condition syntax is:
 - **<“if”> <condition> <“then”> <StandardRequirement>**
 - **Consider the following requirement: “If the **user enters the wrong password**, then the **system shall send an error message to the user**.”** In the case, “**user enters the wrong password**” is the condition. The part after “**then**” is treated like a **standard requirement**.

Week 4 and 5 Lessons: Developing Requirement Analysis



- ❖ **Techniques that should be used to avoid common requirements problems ...**
 - **Business Rules Syntax.** RAT treats all requirements that start with “**all**”, “**only**” and “**exactly**” as **business rules**.
 - An example is: “**Only** the members of **payroll department** will be able to **access the payroll database**”.



Week 4 and 5 Lessons: Developing Requirement Analysis

❖ (C)lass (R)esponsability (C)ollaborators cards.

- Helps to link Requirements to UML Analysis Models.
- A **CRC** is a brainstorming technique for designing interactions in **use cases** by assigning **responsibilities** and **collaborations** for classes.
- A **CRC card** is a card layout for a **single class** and identifies **the class** and its **attributes**, its **responsibilities**, i.e., **functions**, and its **collaborations**, i.e. the **methods** in its **API**.

Week 4 and 5 Lessons:

Developing Requirement Analysis



❖ (C)lass (R)esponsability (C)ollaborators cards.

- Helps to link Requirements to UML Analysis Models.
 - **CRC** is a portable and an anthropomorphic.
- What Is Anthropomorphism? Anthropomorphism is a literary device that assigns human characteristics to nonhuman entities like animals or inanimate objects.
- **Anthropomorphic** means treating animals and objects as if they are human in appearance.

Week 4 and 5 Lessons: Developing Requirement Analysis



❖ (C)lass (R)esponsability (C)ollaborators cards.

- **Example: How to Write Anthropomorphism?**
- **On Sep 21, 2021, written by the MasterClass staff,**
“Some of the most compelling stories we tell as human beings are not about humans at all. For most of human history, people have told stories in which animals or inanimate objects act in human-like ways. The term for this is *anthropomorphism*. As a writer or storyteller, understanding and using anthropomorphism in your work can open up a whole new world of fantastical characters and settings for you to explore.”



Week 4 and 5 Lessons:

Developing Requirement Analysis

❖ What is CRC Card ?

○ Class

- An object-oriented class name.
- Include information about super- and sub-classes.

○ Responsibility

- What information this class stores?
- What this class does?
- The behavior for which an object is accountable.

○ Collaboration

- Relationship to other classes.
- Which other classes this class uses.

❖ CRC Model?

- A CRC Model is a collection of CRC cards.



Week 4 and 5 Lessons: Developing Requirement Analysis

❖ CRC Uses

- CRC used during:
 - Analysis: Looking at problem domain.
 - Design: Refining responsibility.
- Can also be used to:
 - Select core classes
 - Act out class relationships and scenarios
 - Refine project requirement.
 - Assist project management(giving a idea of the number and complexity of the required classes)
 - Serve as a guide to coding (to some degree)

Week 4 and 5 Lessons: Developing Requirement Analysis



❖ CRC ...

- Object oriented systems are modeled using objects which initially reflect the abstractions identified in the problem domain.
- CRC cards capture the object model in terms of classes, responsibilities and collaborations, and the class model in terms of subclass and superclass relationships.
- CRCs are suited to dynamic team work with particular emphasis on role playing. The team member act out the actions of a particular class. CRCs encourage an antropomorphic view.



Week 4 and 5 Lessons: Developing Requirement Analysis

❖ CRC and Use Cases

- With the emphasis on interaction and anthropomorphism CRC are more appropriate for team collaboration. CRC can be used to develop both the class diagram (structural model) and the sequence diagram (dynamic model). Use case focus more on the requirements from the actors perspective.

Week 4 and 5 Lessons: Developing Requirement Analysis

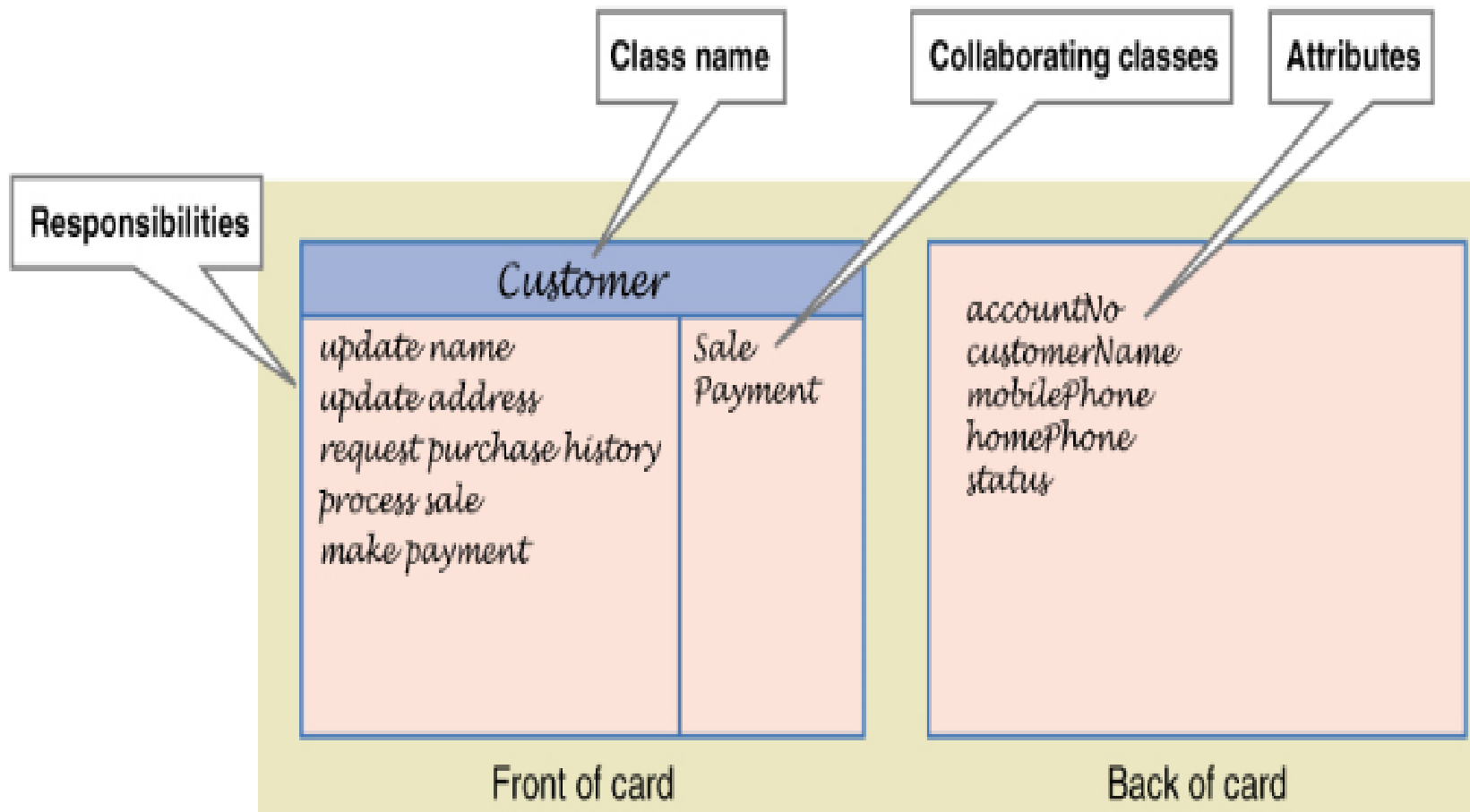


❖ CRC and Use Cases

- The examination of use cases on its own is not a good way to find objects and classes. A class model is also required, techniques such as CRC cards are still useful. For example, a customer may never directly interact with a sales system (say its done by a sales person), however a customer would certainly be a class in a sales system. Also other concepts such as architecture need to be developed.

Week 4 and 5 Lessons: Developing Requirement Analysis

❖ Example of CRC Card



Week 4 and 5 Lessons:

Developing Requirement Analysis



❖ Case Study (Requirement)

- A bank plans a new to build ATM system using OO methods. The ATM system will interface with the customers through a display screen, numeric and special input keys, a bank card reader, a deposit slot, and a receipt printer. Customers may make deposits, withdrawals, and balance enquiries using the ATM machine but the update of accounts will be handled by an interface to the Accounts system. Customers will be assigned a PIN and a clearance level by the Security system which can be verified prior to transactions. In the future we would also like to allow customers to update routine information such as change of address or phone number using the ATM.

Week 4 and 5 Lessons: Developing Requirement Analysis



❖ **ATM candidate Classes**

- ATM CashDispencer
- Financial Screen Message
Tranasction Display



Week 4 and 5 Lessons: Developing Requirement Analysis

○ Example of CRC forms

Class name : Bank	
Superclass : nil	
Role : Validates transactions and Customers by communicating with the Bank's central accounts database.	
Attributes : customers, accounts	
Responsibilities	Collaborators
Validate a Customer	Card, PIN
Validate a withdraw transaction	Card, Account



Week 4 and 5 Lessons: Developing Requirement Analysis

○ Example of CRC forms ...

Class name : Withdrawal	
Superclass : Transaction	
Role : Store withdraw transaction information	
Attributes : date, time, account, card Id, ATM Id, amount	
Responsibilities	Collaborators
Knowing date of transaction	Date
Knowing time of transaction	Time
Knowing amount of transaction	Money
Knowing card used in transaction	Card

Week 4 and 5 Lessons: Developing Requirement Analysis

○ Example

scenario : ATM Verify Customer

System or Component : ATM

Scenario : Verify Customer

Extends :

Actor(s) : Customer		Alternates : 1. Card invalid 2. PIN incorrect 3. Customer cancels process	
Precondition : ATM is open (Scenario "Open ATM" has been carried out)			
Step :	Source :	Action :	Comments :
1	User	inserts card into card reader.	
2	ATM	checks that card is valid for this ATM.	card is valid
3	ATM	display request : "enter PIN number then press OK key".	
4	User	enters PIN using the numeric keys and presses the OK key.	correct PIN is entered
5	ATM	verifies that card and PIN are valid with bank.	verification successful
6	ATM	logs the customer connection.	
7	ATM	display request : "select transaction type using the transaction keys or press the Cancel key to discontinue use of ATM".	